

RAG in LangChain

1. What is RAG?

- RAG = Retrieval-Augmented Generation
- RAG is a technique where an AI model first **retrieves relevant information from external data** and then uses that information to **generate an answer**.

2. Why Do We Use RAG?

- An LLM has limitations:
 - 1) It may not know your **private data**.
 - 2) It may not know your **company documents**.
 - 3) It may not know your **latest information**.
 - 4) It may give incorrect answers when the required information is not in its knowledge.
- RAG allows us to provide our own data to the LLM.

College Rules PDF

Student Handbook PDF

Exam Rules PDF

Fee Structure PDF

3. Where Do We Use RAG?

- 1) Chat with PDF
- 2) College/University chatbot
- 3) Company knowledge chatbot
- 4) Customer support chatbot
- 5) Documentation chatbot
- 6) Legal document search
- 7) Healthcare information systems
- 8) Product information chatbot

4. What Do We Need to Learn for RAG?

- 1) Document Loaders
- 2) Text Splitters
- 3) Embeddings

- 4) Vector Database / Vector Store
- 5) Retriever
- 6) Prompt + LLM
- 7) RAG Chain

1. LangChain Document Loaders

1) What is a Loader?

- A Document Loader in LangChain is a component used to load data from different sources and convert it into a standard **Document format** so that LangChain can process it.

- 1) TextLoader
- 2) PyPDFLoader
- 3) WebBaseLoader
- 4) CSVLoader

2) TextLoader

- TextLoader is used to load text data from a .txt file into LangChain.

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("demo.txt")

documents = loader.load()

print(documents)

# To see only the text:
print(documents[0].page_content)
```

3) PyPDFLoader

- PyPDFLoader is used to load and extract text from PDF files.

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("demo.pdf")
```

```
documents = loader.load()

print(documents)
```

4) WebBaseLoader

- WebBaseLoader is used to load and extract text/content from a web page.
- pip install beautifulsoup4
- pip install requests

```
from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader(
    "https://example.com"
)

documents = loader.load()

print(documents[0].page_content)
```

5) CSVLoader

- CSVLoader is used to load data from a CSV file and convert the rows into LangChain Documents.

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader("students.csv")

documents = loader.load()

print(documents)
```

2. What is a Text Splitter?

- When we load a large document using a loader, we may get a **very large amount of text**. Instead of processing the entire document at once, we divide it into **smaller pieces called chunks**.

Why Do We Use Text Splitters?

- Large documents are divided into manageable pieces.
- Relevant information can be retrieved more easily.
- Chunks can be converted into embeddings.
- Chunks can be stored in a vector database.
- RAG systems become more efficient.

Where is Text Splitting Used?

Text splitting is commonly used in:

- Chat with PDF
- RAG applications
- Document Q&A
- Company knowledge chatbot
- Website chatbot
- Documentation search
- Semantic search

3. Types of Text Splitters

- 1) Length-Based Splitting
- 2) Text-Structure-Based Splitting
- 3) Document-Structure-Based Splitting
- 4) Semantic-Meaning-Based Splitting

1) Length-Based Text Splitting

- The text is divided according to a maximum character/token length.

```
from langchain_text_splitters import CharacterTextSplitter

text = """
Python is a programming language.
It is easy to learn.
Python is used in web development.
Python is also used in AI and data science.
"""

splitter = CharacterTextSplitter(
    chunk_size=50,
    chunk_overlap=0
)

chunks = splitter.split_text(text)

for chunk in chunks:
    print(chunk)
    print("-----")
```

2) Text-Structure-Based Splitting

- Here we don't simply cut text randomly by length.
- We try to split the text using **text separators**, such as:

Paragraph

Sentence

Word

Character

Recursive splitter split based on

1. \n\n → paragraph break

2. \n → line break

3. " " → space

4. "" → individual characters

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text = """
Python is a programming language.

Python is easy to learn and widely used.

Python is used in AI, web development and data science.
"""

splitter = RecursiveCharacterTextSplitter(
    chunk_size=100,
    chunk_overlap=20
)

chunks = splitter.split_text(text)

for chunk in chunks:
    print(chunk)
    print("-----")
```

3) Document-Structure-Based Splitting

- Some documents already have a **clear structure**.
- For example, a Markdown document may contain:

```
from langchain_text_splitters import MarkdownHeaderTextSplitter

text = """
# Python

Python is a programming language.

## Features

Python is easy to learn.

## Applications
```

```

Python is used in AI and web development.
"""

headers_to_split_on = [
    ("#", "Heading 1"),
    ("##", "Heading 2")
]

splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=headers_to_split_on
)

documents = splitter.split_text(text)

for doc in documents:
    print(doc.page_content)
    print(doc.metadata)
    print("-----")

```

4) Semantic Meaning-Based Splitting

- Semantic splitting divides text into chunks based on changes in the meaning or semantic similarity of the content.

```

from langchain_experimental.text_splitter import SemanticChunker
from langchain_google_genai import GoogleGenerativeAIEmbeddings

embeddings = GoogleGenerativeAIEmbeddings(
    google_api_key="YOUR_GOOGLE_API_KEY",
    model="models/gemini-embedding-001"
)

splitter = SemanticChunker(embeddings)

text = """
Python is a programming language.
Python is easy to learn.
Python is widely used in software development.

The stock market changes every day.
Investors buy and sell shares.

```

```
Stock prices can increase or decrease.
"""

documents = splitter.create_documents([text])

for doc in documents:
    print(doc.page_content)
    print("-----")
```